

Robot Learning

Lecture 5 Model-Free RL -- Value-based

Refresh of prev. lecture

Model-Free Prediction

MC Vs TD Learning

- Goal: learn v_π from episodes of experience under policy π
- Incremental every-visit MC
 - Update value $V(S_t)$ toward actual return G_t

$$V(S_t) \leftarrow V(S_t) + \alpha(G_t - V(S_t))$$

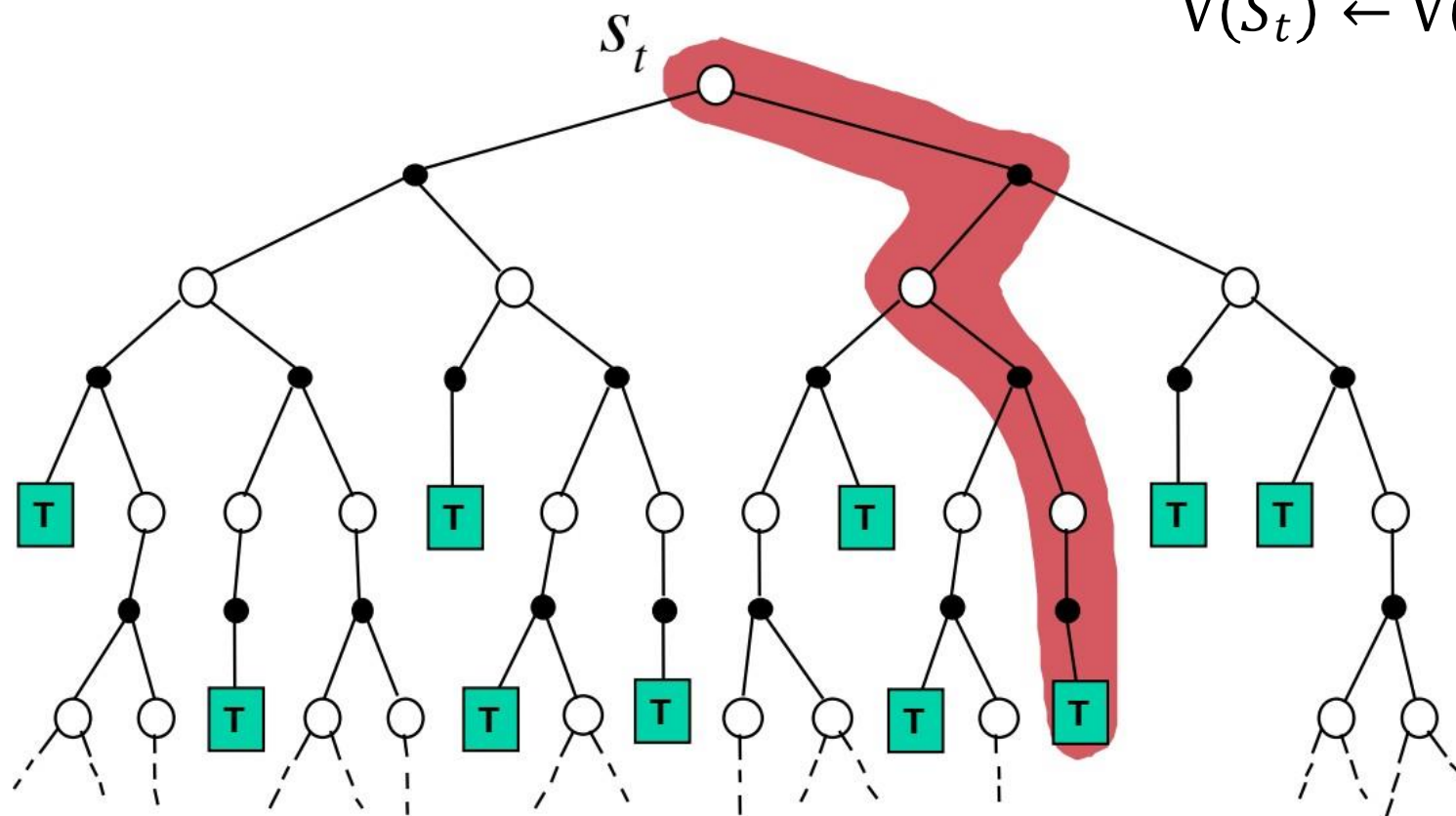
- Simplest temporal-difference learning algorithm (TD(0))
 - Update value $V(S_t)$ toward estimated return $R_{t+1} + \gamma V(S_{t+1})$

$$V(S_t) \leftarrow V(S_t) + \alpha \underbrace{(R_{t+1} + \gamma V(S_{t+1}))}_{\text{TD target}} - V(S_t)$$

$\underbrace{\hspace{15em}}_{\text{TD error } \delta_t}$

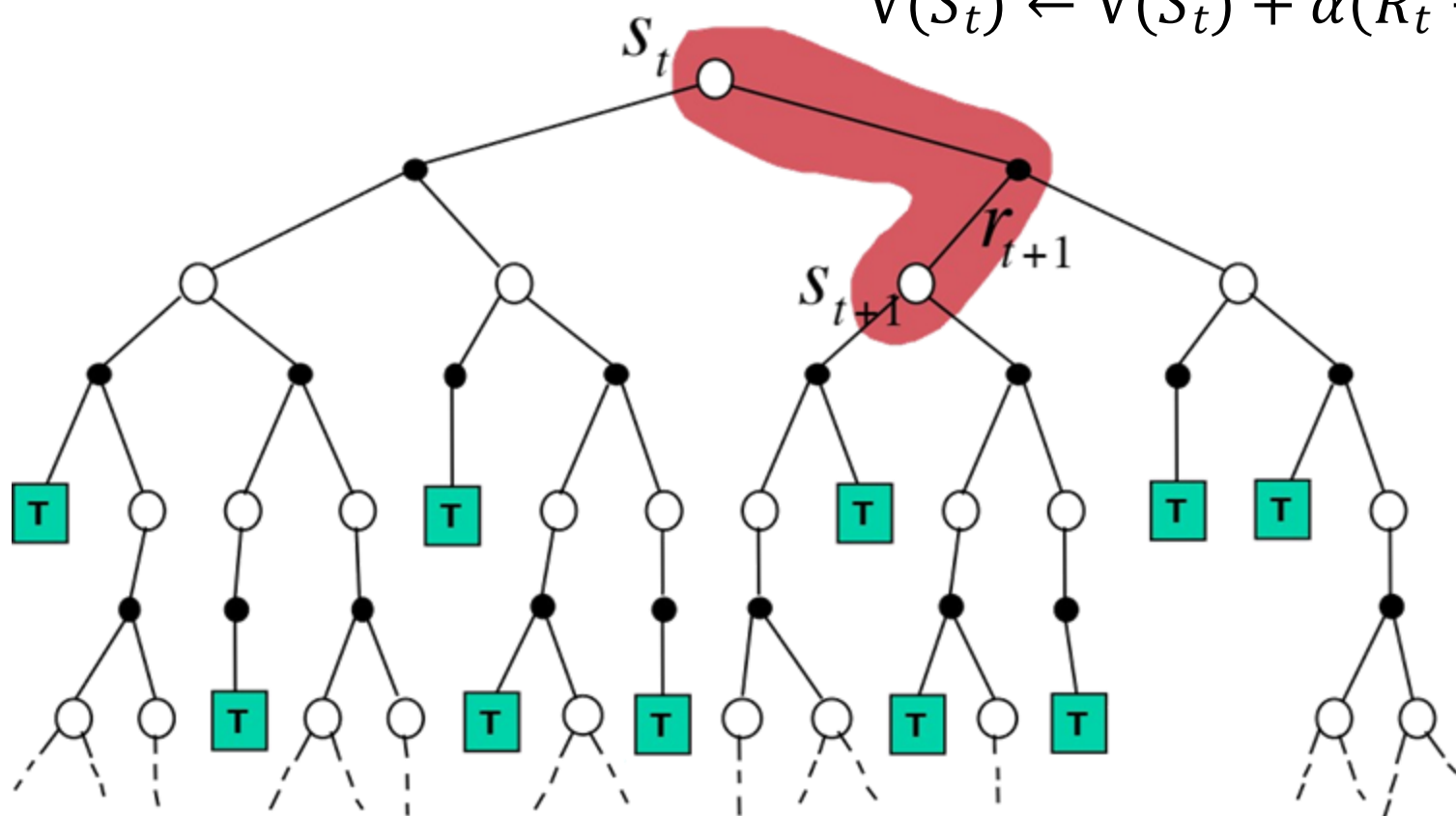
MC Update

$$V(S_t) \leftarrow V(S_t) + \alpha(G_t - V(S_t))$$

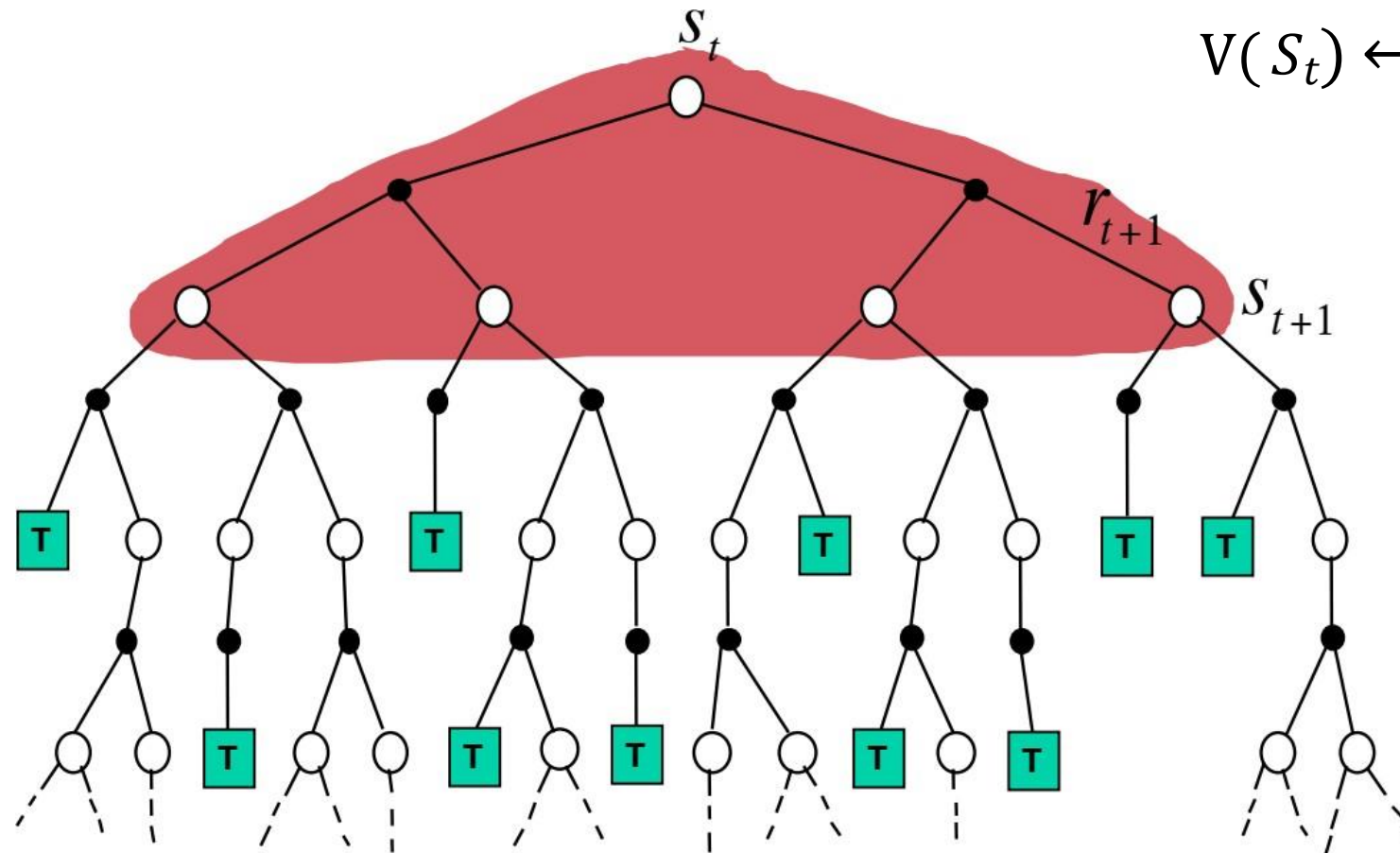


TD Update

$$V(S_t) \leftarrow V(S_t) + \alpha(R_t + \gamma V(S_{t+1}) - V(S_t))$$



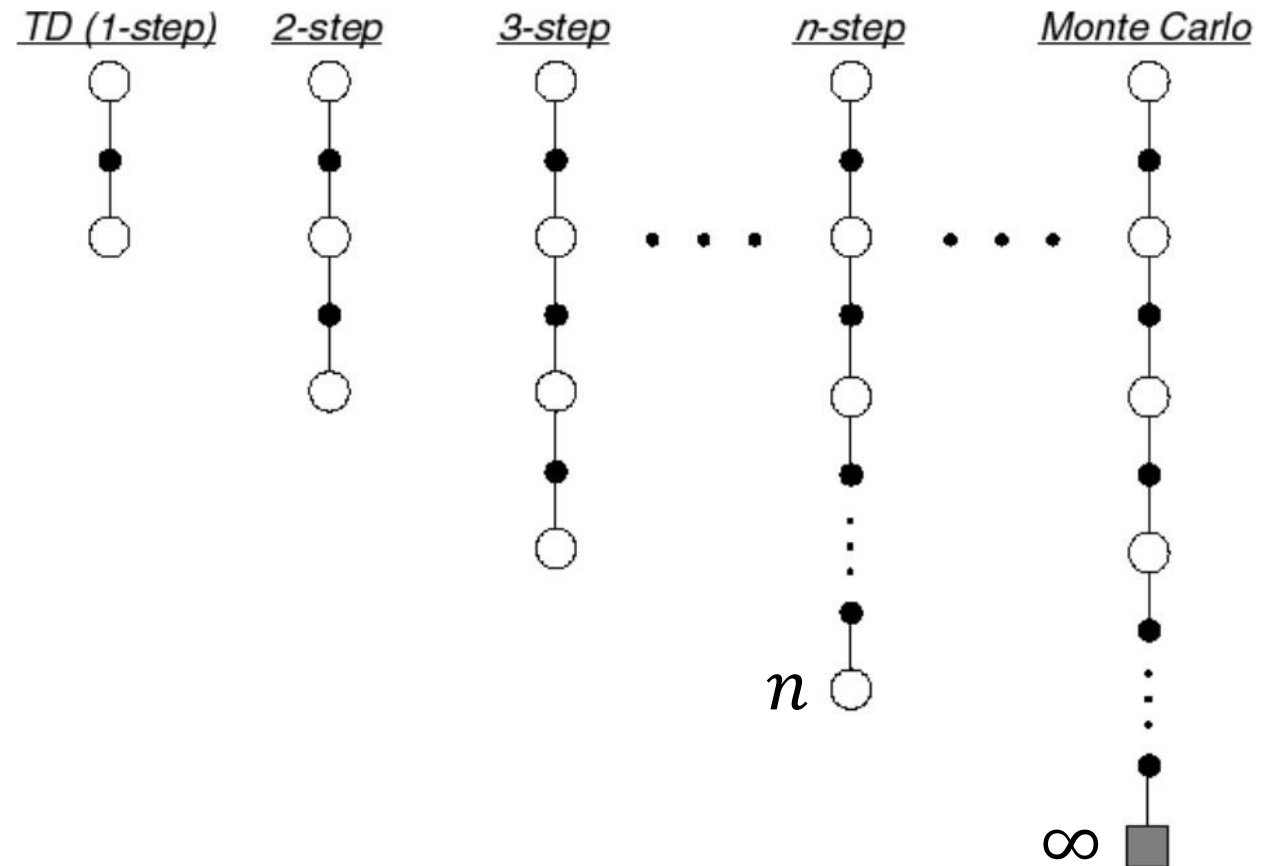
Dynamic Programming



$$V(S_t) \leftarrow \mathbb{E} [R_{t+1} + \gamma V(S_{t+1})]$$

n -step Prediction

Have TD look and
target n steps in the
future

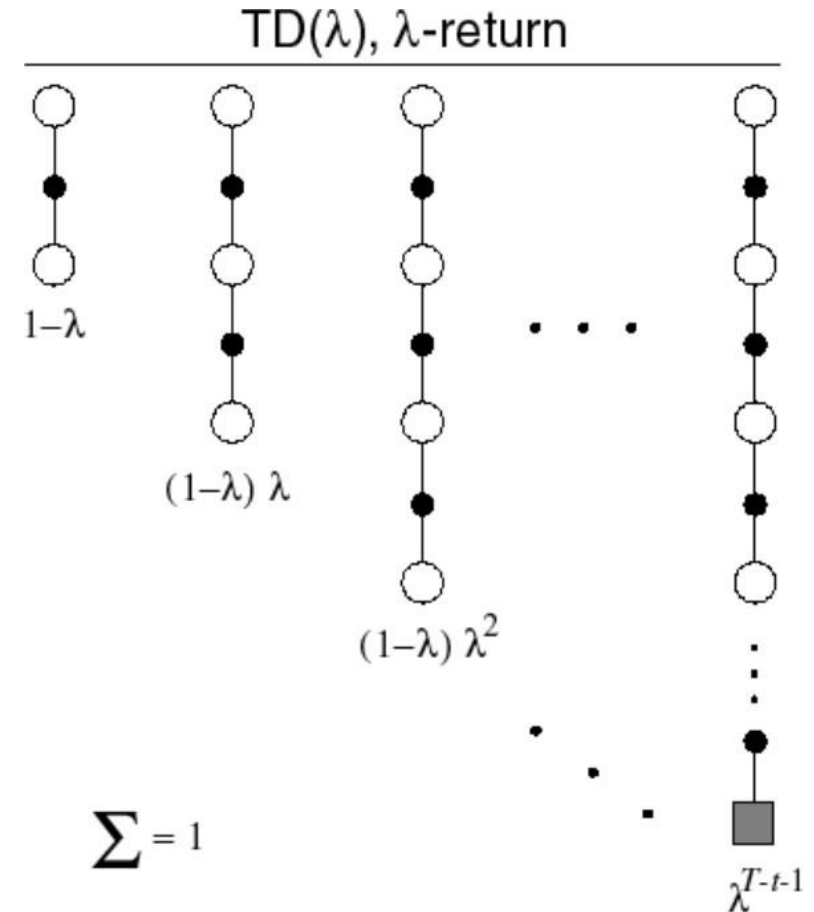


λ -returns

- The λ -return G_t^λ combines all n -step returns $G_t^{(n)}$
- Using weight $(1 - \lambda)\lambda^{n-1}$

$$G_t^\lambda = (1 - \lambda) \sum_{n=1}^{\infty} \lambda^{n-1} G_t^{(n)}$$

- Update as appropriate (TD(λ))
 $V(S_t) \leftarrow V(S_t) + \alpha(G_t^\lambda - V(S_t))$



Backward View TD(λ)

- ✓ Forward view provides theory
- ✓ Backward view provides mechanism
- ✓ Update online, every step, from incomplete sequences

Eligibility Traces

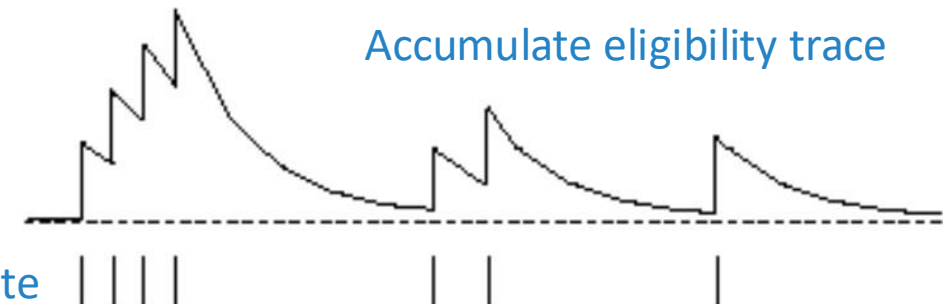


- Credit assignment problem: what caused shock?
 - **Frequency heuristic**: assign credit to most frequent states
 - **Recency heuristic**: assign credit to most recent states
- **Eligibility** traces **combine both** heuristics

$$E_0(s) = 0$$

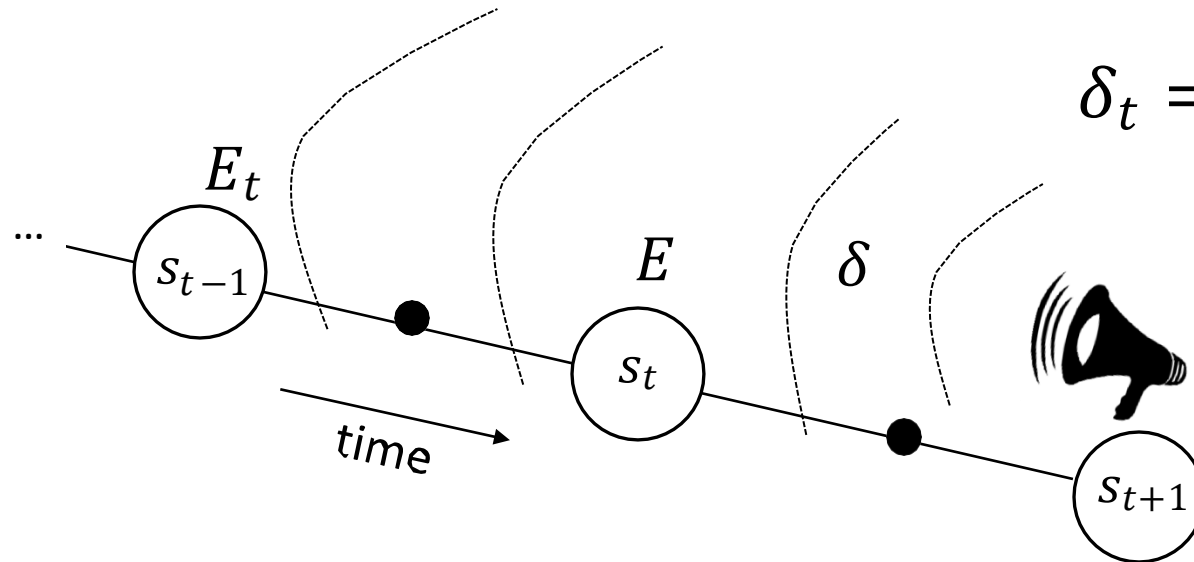
$$E_t(s) = \gamma \lambda E_{t-1}(s) + \mathbf{1}(S_t; s)$$

times of visit to state



Backward View TD(λ)

- Keep an **eligibility trace** for every state s
- Update value $V(s)$ for every state s
- In proportion to **TD-error** δ_t and **eligibility trace** $E_t(s)$

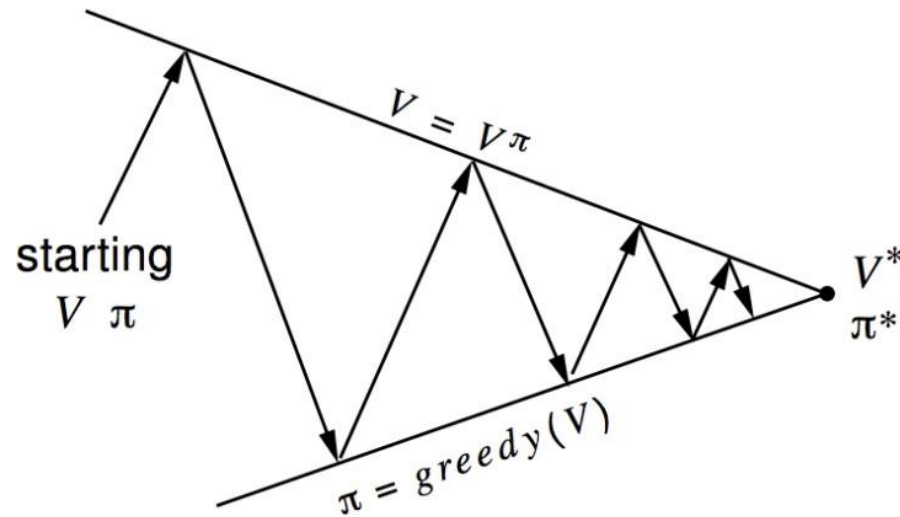


$$\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$$

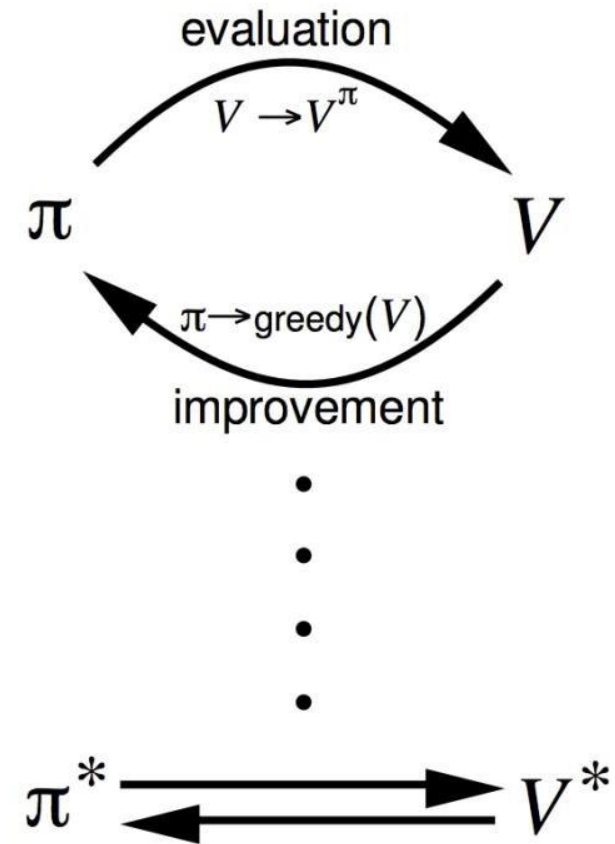
$$V(s) = V(s) + \alpha \delta_t E(s)$$

Model-Free Control

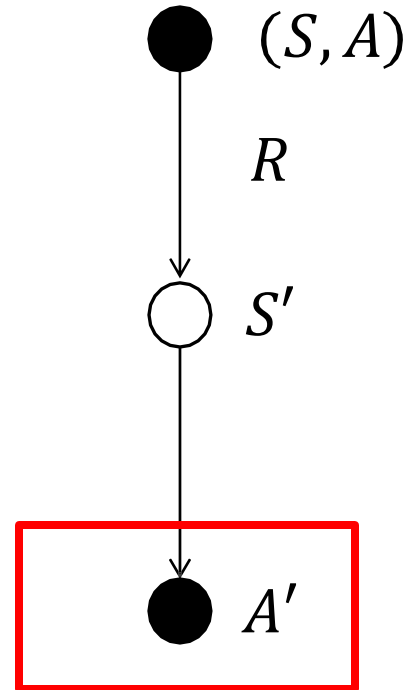
Generalized Policy Iteration (Lecture 5b)



- **Policy evaluation** - Estimate v_π
 - **Any** policy evaluation
- **Policy improvement** - Generate $\pi' \geq \pi$
 - **Any** policy improvement algorithm



Updating Action-Value Functions with SARSA



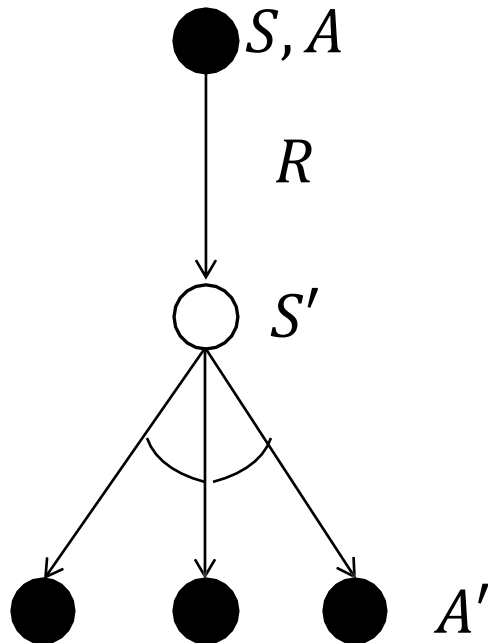
Expected SARSA

$$q_*(s, a) = \mathcal{R}_s^a + \gamma \sum_{a' \in \mathcal{A}} \pi(a'|s') q_*(s', a')$$

We sample also future action A'
(instead of leveraging policy to compute
expectation)

$$Q(S, A) \leftarrow Q(S, A) + \alpha(R + \gamma Q(S', A') - Q(S, A))$$

Q-Learning Control Algorithm

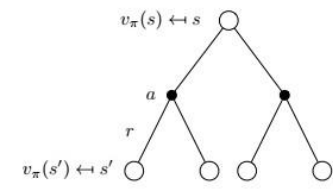
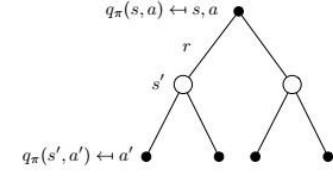
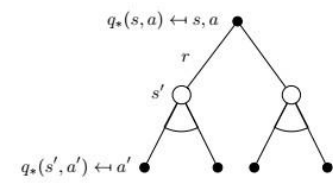
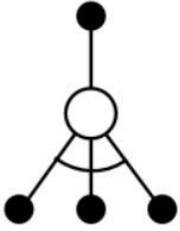


Theorem

Q-learning control converges to the optimal action-value function, $Q(s, a) \rightarrow q_*(s, a)$

$$Q(S, A) \leftarrow Q(S, A) + \alpha \left(R + \max_{a'} \gamma Q(S', a') - Q(S, A) \right)$$

Dynamic Programming Vs Temporal Difference Learning

	<i>Full Backup (DP)</i>	<i>Sample Backup (TD)</i>
Bellman Expectation Equation for $v_{\pi}(s)$	 Iterative Policy Evaluation	 TD Learning
Bellman Expectation Equation for $q_{\pi}(s, a)$	 Q-Policy Iteration	 Sarsa
Bellman Optimality Equation for $q_{*}(s, a)$	 Q-Value Iteration	 Q-Learning

Lecture 5d:

Model-Free Reinforcement Learning – Value Function Approximation

Slides heavily adapted from David Silver (UCL)

Outline

- Introduction
- Incremental methods
 - Stochastic Gradient Descent
 - Gradient TD learning
 - Linear value approximation
- Batch methods
 - Linear least squares
 - Experience replay & DQN

Introduction

Reinforcement Learning on the Scale

How to use RL when state space is not trivial?

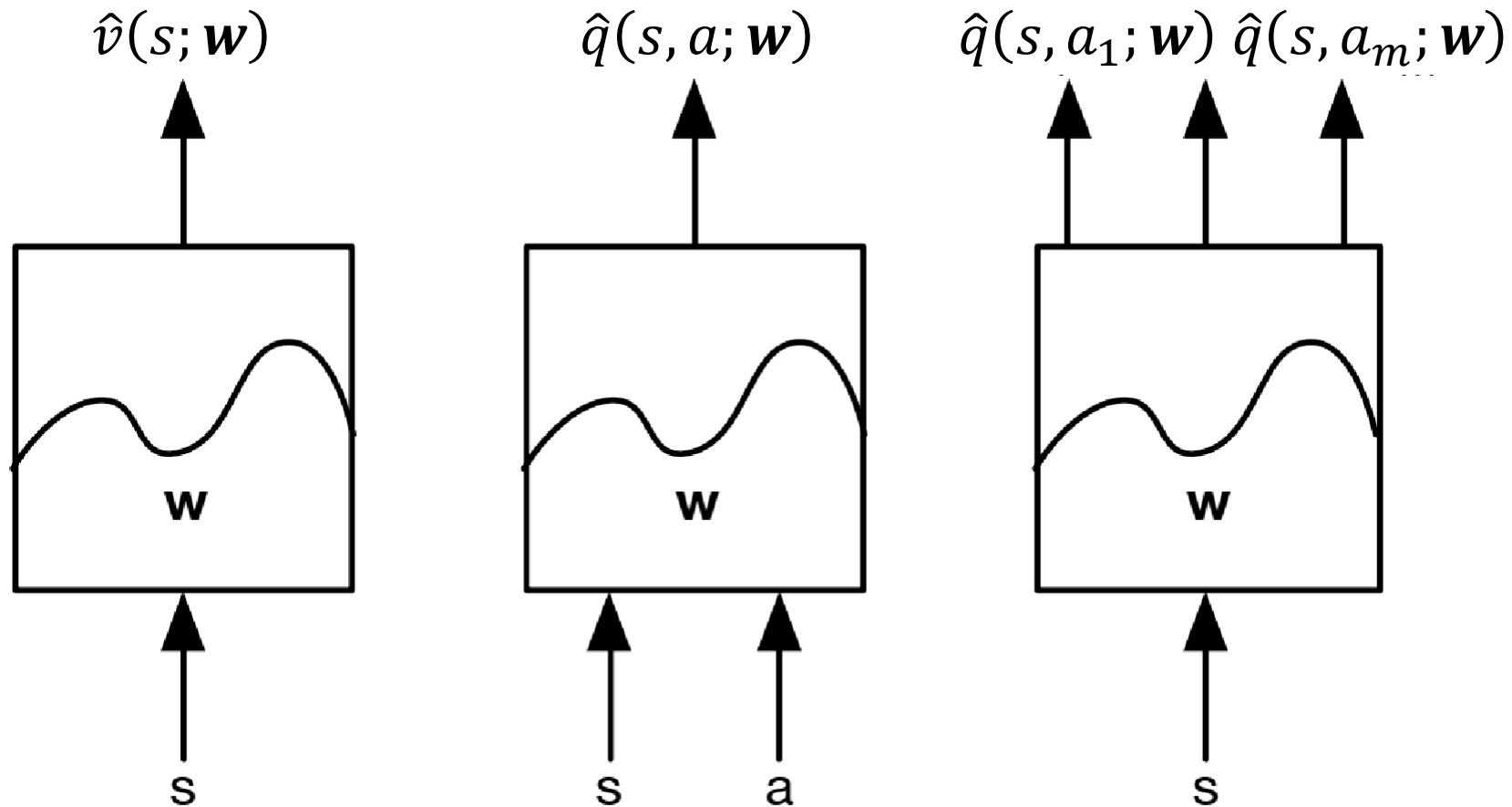
- Backgammon: 1020 states
- Go: 10170 states
- Robot control: continuous state space
- Molecule search: $>10^{60}$ states

Scale up model-free methods for *prediction* and *control* from the last lecture?

Value Function Approximation

- So far $V(s)/Q(s, a) =$ **lookup table**
 - An entry for every state s or state-action pair s, a
 - Large MDPs $\Rightarrow$ too **many states and/or actions** to store in memory
 - Too **slow to learn** the value of each state individually
 - **Generalization** issues
- The *new* approach
 - Estimate value function with **function approximation**
$$\hat{v}(s; \mathbf{w}) = v_{\pi}(s)$$
$$\hat{q}(s, a; \mathbf{w}) = q_{\pi}(s, a)$$
 - Generalise from seen states to **unseen states**
 - Update parameters **$\mathbf{w}$** using **MC or TD learning** and following Bellman principle

Value Function Approximation Approaches



Which Function Approximator?

- Linear combinations of features
- Neural network
- Decision tree
- Nearest neighbour
- Fourier / wavelet bases
- ...

Which Function Approximator?

- Linear combinations of features
- Neural network
- Decision tree
- Nearest neighbour
- Fourier / wavelet bases
- ...



Focus on differentiable
methods

In addition, we require training methods suitable for

- non-stationary data (e.g. because I am changing the policy, or env is changing)
- non-iid data (because data are result of experience which are a series of sequential decisions)

Incremental Methods

Stochastic Gradient Descent in Value Approximation

- Goal: find a parameter vector $\mathbf{w}$ that minimizes the mean-square error between approximated $\hat{v}(s; \mathbf{w})$ and true $v_\pi(s)$ value function

$$J(\mathbf{w}) = \mathbb{E}_\pi[(v_\pi(S) - \hat{v}(S; \mathbf{w}))^2]$$

Let's pretend to have an oracle

- The gradient of $J(\mathbf{w})$ is:

$$\Delta \mathbf{w} = -\frac{1}{2} \alpha \nabla_{\mathbf{w}} J(\mathbf{w}) = \alpha \mathbb{E}_\pi[v_\pi(S) - \hat{v}(S; \mathbf{w})] \nabla_{\mathbf{w}} \hat{v}(S; \mathbf{w})$$

- Stochastic approx. (sampling)

$$\Delta \mathbf{w} = \alpha (v_\pi(S) - \hat{v}(S; \mathbf{w})) \nabla_{\mathbf{w}} \hat{v}(S; \mathbf{w})$$

Get $v_\pi(S)$ from sampling

Feature Vector State

- Represent state by a feature vector

$$x(S) = \begin{bmatrix} x_1(S) \\ \vdots \\ x_n(S) \end{bmatrix}$$

For example:

- Robot distance from landmarks
- Trends in the stock market
- Piece configurations in chess
- Neural embedding

Linear Value Function Approximation

- Value as a linear combination of state features

$$\hat{v}(S; \mathbf{w}) = x(S)^T \mathbf{w}$$

- Objective function is quadratic in parameters $\mathbf{w}$
 - Stochastic gradient descent converges to global optimum
 - Simple (LMS) update rule

$$\begin{aligned}\nabla_{\mathbf{w}} \hat{v}(S; \mathbf{w}) &= x(S) \\ \Delta \mathbf{w} &= \alpha (v_{\pi}(S) - \hat{v}(S; \mathbf{w})) x(S)\end{aligned}$$

I'm just moving linearly
towards the minimization of the
error w.r.t. an oracle

Lookup Table & Features

- A special case of linear value function approximation
- Using table **lookup features**

$$x(S) = \begin{bmatrix} \mathbf{1}(S; s_1) \\ \vdots \\ \mathbf{1}(S; s_n) \end{bmatrix}$$

One hot encoding of states

- Parameter vector $\mathbf{w}$ values each state

$$\hat{v}(S; \mathbf{w}) = \begin{bmatrix} \mathbf{1}(S; s_1) \\ \vdots \\ \mathbf{1}(S; s_n) \end{bmatrix}^T \begin{bmatrix} w_1 \\ \vdots \\ w_n \end{bmatrix}$$

Incremental Prediction

Incremental Prediction Algorithms

- So far have assumed access to true $v_\pi(s)$, but in RL there is **no supervisor, only rewards**
- In practice, we substitute a target for $v_\pi(s)$

- MC - Target is the return G_t

$$\Delta \mathbf{w} = \alpha(\mathbf{G}_t - \hat{v}(S_t; \mathbf{w})) \nabla_{\mathbf{w}} \hat{v}(S_t; \mathbf{w})$$

- TD(0) - Target is the TD target $R_{t+1} + \gamma \hat{v}(S_{t+1}; \mathbf{w})$

$$\Delta \mathbf{w} = \alpha(\mathbf{R}_{t+1} + \gamma \hat{v}(S_{t+1}; \mathbf{w}) - \hat{v}(S_t; \mathbf{w})) \nabla_{\mathbf{w}} \hat{v}(S_t; \mathbf{w})$$

Reward of getting the action, plus the value of next state

- TD(λ) - Target is the λ -return G^λ

$$\Delta \mathbf{w} = \alpha(\mathbf{G}_t^\lambda - \hat{v}(S_t; \mathbf{w})) \nabla_{\mathbf{w}} \hat{v}(S_t; \mathbf{w})$$

Of note, I am updating $\hat{v}$ using $\hat{v}$ -- new knowledge comes from R_{t+1}

Value Function Approximation - MC

- Return G_t is an **unbiased, noisy sample** of true value $v_\pi(S_t)$
- We can therefore apply supervised learning to training samples $\langle S_1, G_1 \rangle, \langle S_2, G_2 \rangle, \langle S_1, G_3 \rangle$
- Linear Monte-Carlo policy evaluation
$$\Delta \mathbf{w} = \alpha (G_t - \hat{v}(S_t; \mathbf{w})) \nabla_{\mathbf{w}} \hat{v}(S_t; \mathbf{w}) = \alpha (G_t - \hat{v}(S_t; \mathbf{w})) x(S_t)$$
- Monte-Carlo evaluation converges to a local optimum
- Even when using non-linear value function approximation

Value Function Approximation – TD

- TD-target is a **biased sample** of true value $v_{\pi}(S_t)$
- Can still apply **supervised learning to training samples**
 $\langle S_1, R_2 + \gamma \hat{v}(S_2; \mathbf{w}) \rangle, \dots, \langle S_{T-1}, R_T \rangle$
- **Linear TD(0)** policy evaluation
$$\Delta \mathbf{w} = \alpha (R + \gamma \hat{v}(S'; \mathbf{w}) - \hat{v}(S; \mathbf{w})) \nabla_{\mathbf{w}} \hat{v}(S; \mathbf{w}) = \alpha \delta \mathbf{x}(S)$$
- Linear TD(0) converges (close) to global optimum

Value Function Approximation - TD(λ)

- λ -return G^λ is also a **biased sample** of true value $v_\pi(S_t)$
- Still **supervised learning to train from samples**

$$\langle S_1, G_1^\lambda \rangle, \dots, \langle S_{T-1}, G_{T-1}^\lambda \rangle$$

- **Forward** view linear TD(λ)

$$\Delta \mathbf{w} = \alpha \left(G_t^\lambda - \hat{v}(S_t; \mathbf{w}) \right) \nabla_{\mathbf{w}} \hat{v}(S_t; \mathbf{w}) = \alpha \left(G_t^\lambda - \hat{v}(S_t; \mathbf{w}) \right) \mathbf{x}(S_t)$$

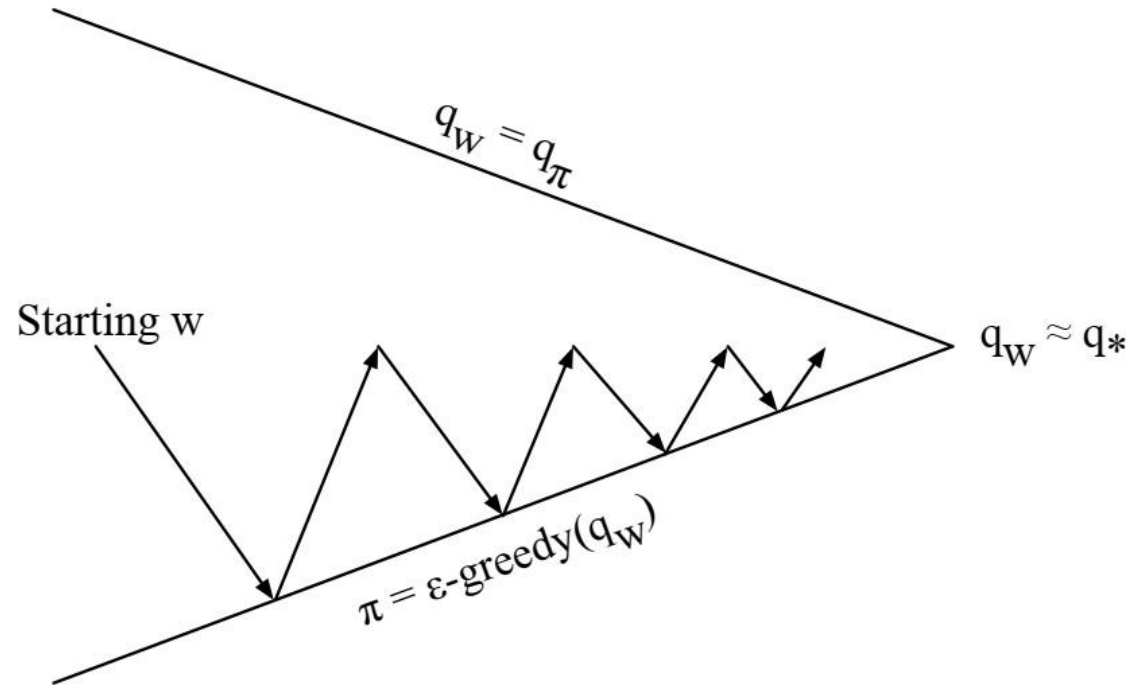
- **Backward** view linear TD(λ)

$$\begin{aligned} \delta_t &= R_{t+1} + \gamma \hat{v}(S_{t+1}; \mathbf{w}) - \hat{v}(S_t; \mathbf{w}) \\ E_t &= \lambda \gamma E_{t-1} + \mathbf{x}(S_t) \\ \Delta \mathbf{w} &= \alpha \delta_t E_t \end{aligned}$$

Notes – Eligibility traces (history of the past + where I am now) so far provided credit for a given state for future rewards. Now not anymore tabular form, hence E gives credit for a proxy of the state (i.e. $\mathbf{x}$, i.e. to the parameters of my model).

Incremental Control

Control with Value Function Approximation



- Policy evaluation - **Approximate** policy evaluation, $\hat{q}(\cdot, \cdot, \mathbf{w}) \approx q_\pi(\cdot, \cdot)$
- Policy improvement - ϵ -greedy policy improvement

Action-Value Function Approximation

- Approximate the action-value function $\hat{q}(S, A, \mathbf{w}) \approx q_\pi(S, A)$
- Minimise MSE between approximate action-value $\hat{q}(S, A, \mathbf{w})$ and true action-value $q_\pi(S, A)$

$$J(\mathbf{w}) = \mathbb{E}_\pi[(q_\pi(S, A) - \hat{q}(S, A, \mathbf{w}))^2]$$

- Use stochastic gradient descent to find a local minimum

$$-\frac{1}{2} \nabla_{\mathbf{w}} J(\mathbf{w}) = (q_\pi(S, A) - \hat{q}(S, A; \mathbf{w})) \nabla_{\mathbf{w}} \hat{q}(S, A; \mathbf{w})$$
$$\Delta \mathbf{w} = \alpha (q_\pi(S, A) - \hat{q}(S, A; \mathbf{w})) \nabla_{\mathbf{w}} \hat{q}(S, A; \mathbf{w})$$

Linear Action-Value Function Approximation

- Use table **feature action-states**

$$x(S, A) = \begin{bmatrix} X_1(S; A) \\ \vdots \\ X_n(S; A) \end{bmatrix}$$

- Represent **action-value function by linear combination** of features

$$\hat{q}(S, A, \mathbf{w}) = x(S, A)^T \mathbf{w}$$

- Stochastic gradient descent **update**

$$\begin{aligned} \nabla_{\mathbf{w}} \hat{q}(S, A, \mathbf{w}) &= x(S, A) \\ \Delta \mathbf{w} &= \alpha (q_{\pi}(S, A) - \hat{q}(S, A, \mathbf{w})) x(S, A) \end{aligned}$$

Incremental Control Algorithms

Also in this case we need a **non-oracular target** for $q_\pi(S, A)$

- MC - Target is the return G_t

$$\Delta \mathbf{w} = \alpha(G_t - \hat{q}(S_t, A_t; \mathbf{w})) \nabla_{\mathbf{w}} \hat{q}(S_t, A_t; \mathbf{w})$$

- TD(0) - Target is the TD target $R_{t+1} + \gamma \hat{q}(S_{t+1}, A_{t+1}; \mathbf{w})$

$$\Delta \mathbf{w} = \alpha(R_{t+1} + \gamma \hat{q}(S_{t+1}, A_{t+1}; \mathbf{w}) - \hat{q}(S_t, A_t; \mathbf{w})) \nabla_{\mathbf{w}} \hat{q}(S_t, A_t; \mathbf{w})$$

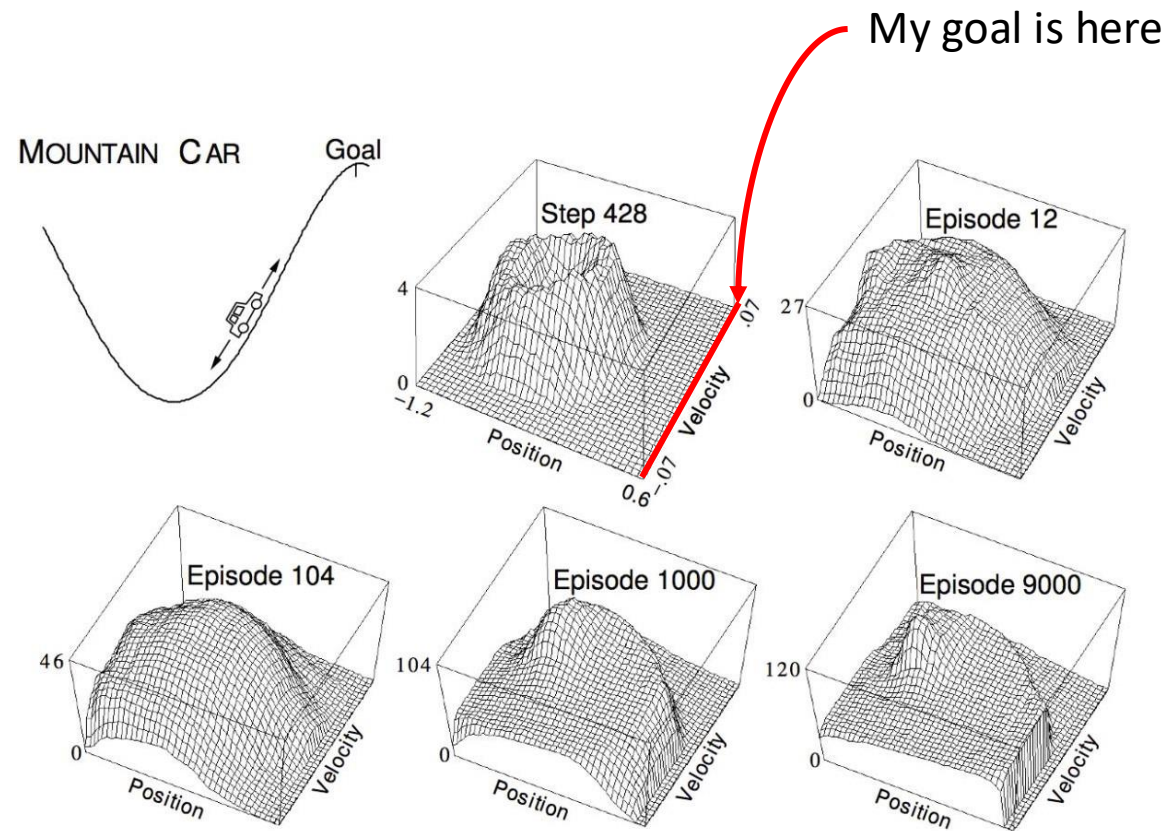
- Forward TD(λ) - Target is the action-value λ -return

$$\Delta \mathbf{w} = \alpha(q_t^\lambda - \hat{q}(S_t, A_t; \mathbf{w})) \nabla_{\mathbf{w}} \hat{q}(S_t, A_t; \mathbf{w})$$

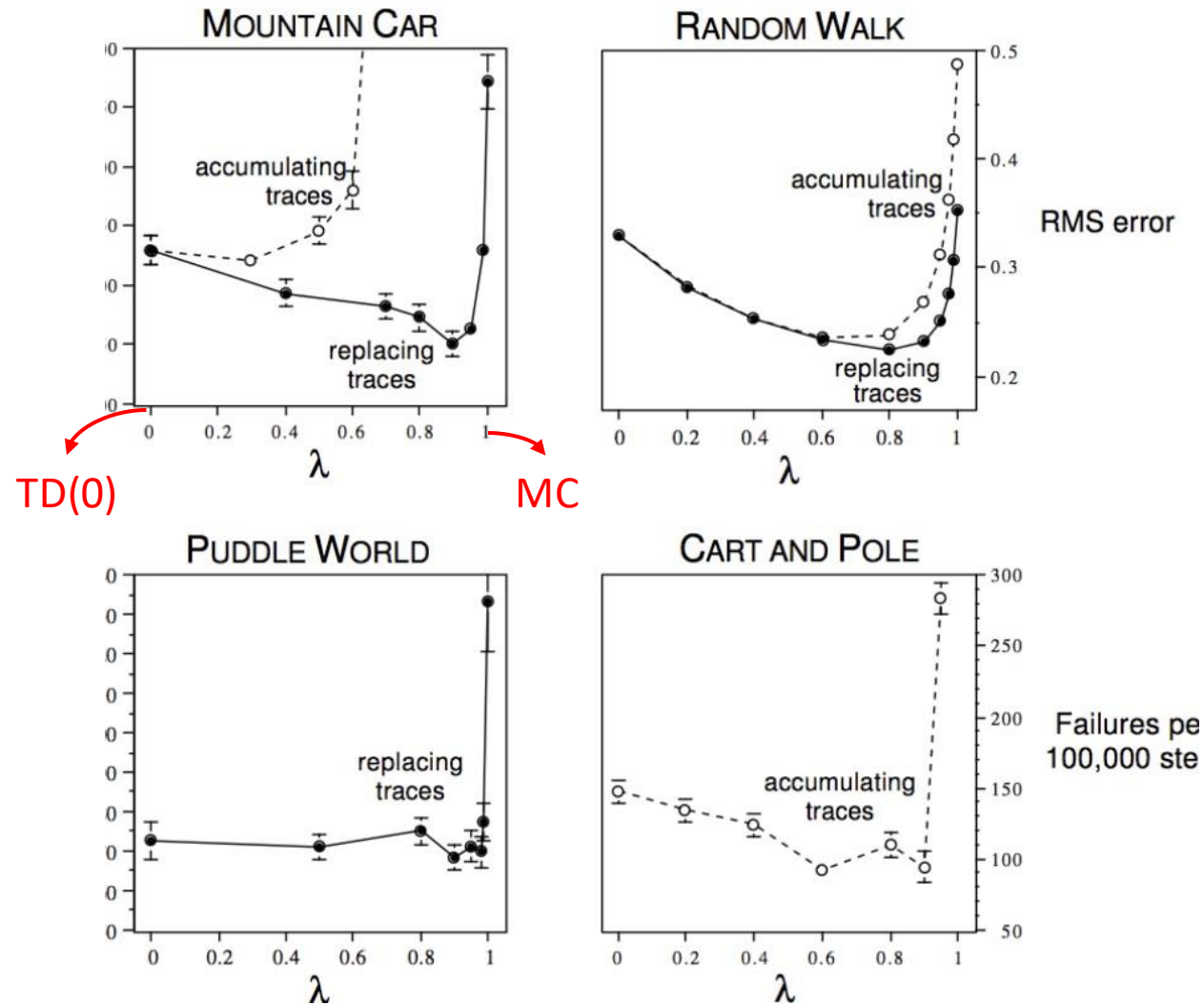
- Backward TD(λ) - Equivalent target

$$\begin{aligned}\delta_t &= R_{t+1} + \gamma \hat{q}(S_{t+1}, A_{t+1}; \mathbf{w}) - \hat{q}(S_t, A_t; \mathbf{w}) \\ E_t &= \lambda \gamma E_{t-1} + \delta_t \\ \Delta \mathbf{w} &= \alpha \delta_t E_t\end{aligned}$$

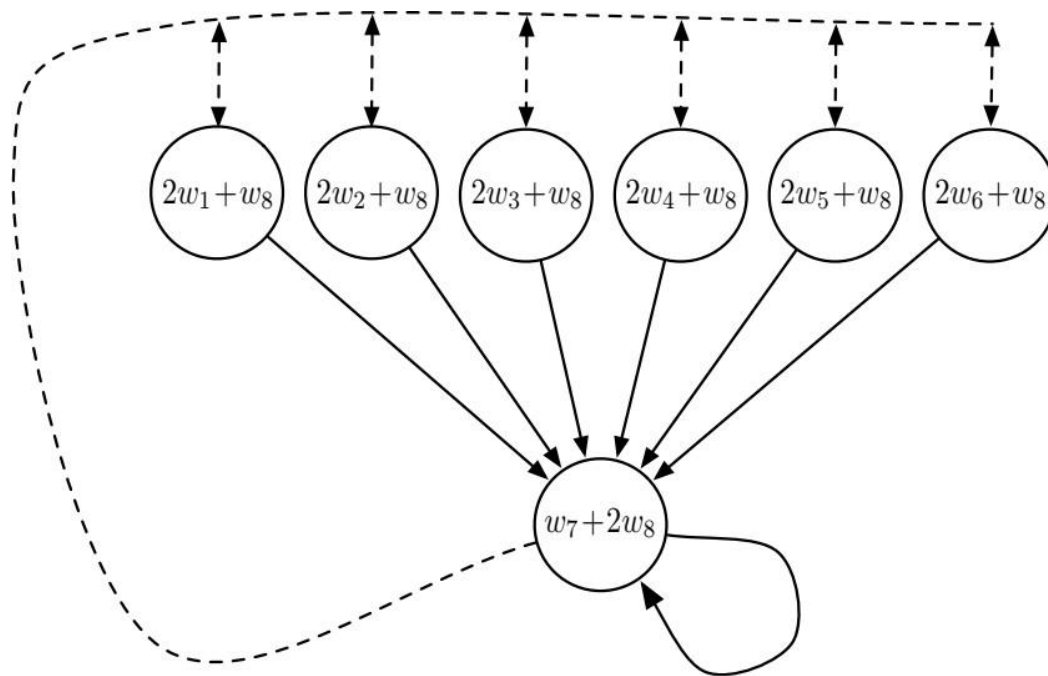
Linear SARSA with Coarse Coding in Mountain Car



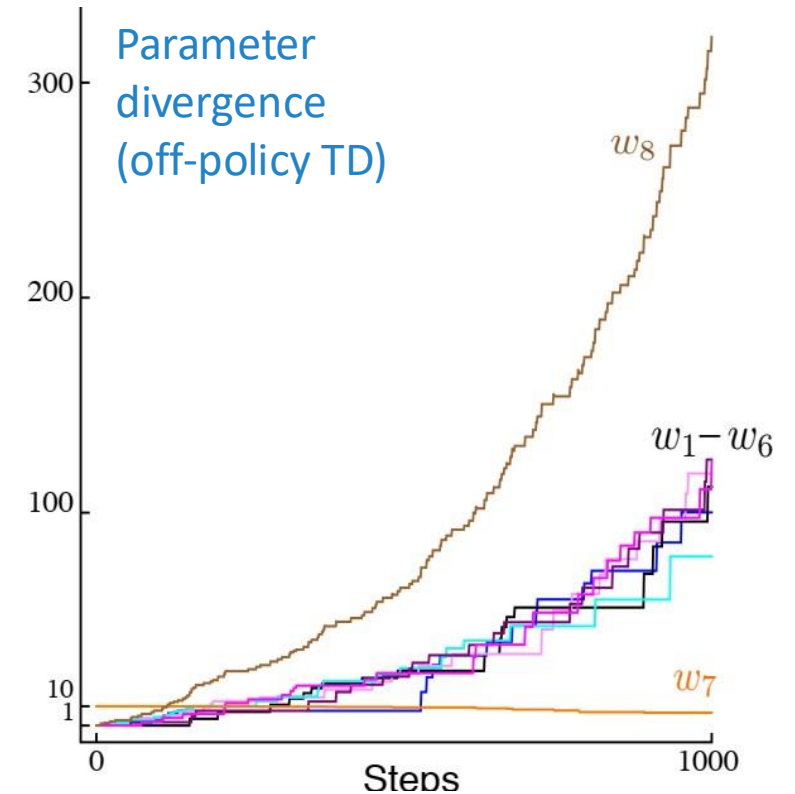
Study of λ : Should We Bootstrap?



Baird's Counterexample (SB Book)



$$\begin{aligned}\pi(\text{solid}|\cdot) &= 1 \\ b(\text{dashed}|\cdot) &= 6/7 \\ b(\text{solid}|\cdot) &= 1/7 \\ \gamma &= 0.99\end{aligned}$$



The Deadly Triad

- Function approximation
- Bootstrapping
- Off policy training

Convergence of Prediction Algorithms

On/Off-Policy	Algorithm	Table	Lookup	Linear	Non-Linear
On-Policy	MC	✓		✓	✓
	TD(0)	✓		✓	✗
	TD(λ)	✓		✓	✗
Off-Policy	MC	✓		✓	✓
	TD(0)	✓		✗	✗
	TD(λ)	✓		✗	✗

Because not biased

Gradient Temporal-Difference Learning

- TD does **not follow the gradient of any objective function**
- TD can diverge when off-policy or using non-linear function approximation
- **Gradient TD** performs SGD in the **projected** Bellman error

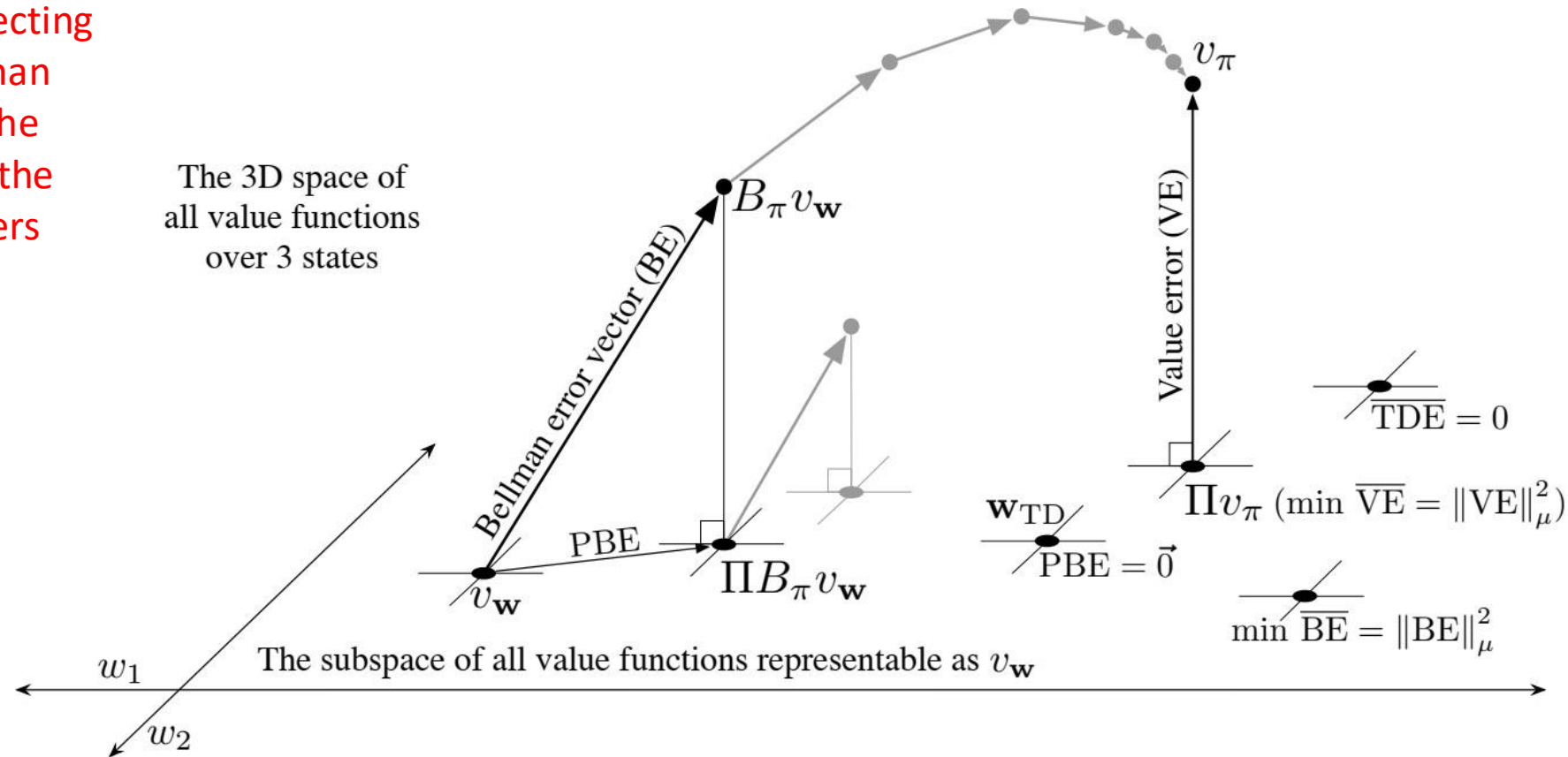
On/Off-Policy	Algorithm	Table Lookup	Linear	Non-Linear
On-Policy	MC	✓	✓	✓
	TD	✓	✓	✗
	Gradient TD	✓	✓	✓
Off-Policy	MC	✓	✓	✓
	TD	✓	✗	✗
	Gradient TD	✓	✓	✓

Projected Bellman Error

Recall: Bellman Error

$$e = \mathbb{E}[R_{t+1} + \gamma v(S_{t+1}) | S_t = s] - v(s)$$

Just projecting
the bellman
error in the
space of the
parameters



Convergence of Control Algorithms

Algorithm	Table Lookup	Linear	Non-Linear
Monte-Carlo Control	✓	(✓)	✗
Sarsa	✓	(✓)	✗
Q-learning	✓	✗	✗
Gradient Q-learning	✓	✓	✗

(✓) = chatters around near-optimal value function

Batch Methods

Batch Reinforcement Learning

- Gradient descent is simple and appealing
- But it is **not sample efficient** -- after you use a sample, that experience is gone
- Batch methods seek to find the **best fitting value function**
- Given the agent's experience (**training data**)

Least Squares Prediction

- Given value function approximation $\hat{v}(s; \mathbf{w}) \approx v_\pi(s)$ and experience $\mathcal{D}$ consisting of *state, value* pairs
$$D = \{\langle S_1, v_1^\pi \rangle, \langle S_2, v_2^\pi \rangle, \dots, \langle S_T, v_T^\pi \rangle\}$$
- Which parameters $\mathbf{w}$ give the best fitting value function $\hat{v}(s; \mathbf{w})$?
- Least squares algorithms find parameter vector $\mathbf{w}$ minimising sum-squared error between $\hat{v}(s; \mathbf{w})$ and the target value $v_\pi(s)$

$$LS(w) = \sum_{t=1}^T (v^\pi - \hat{v}(s_t; \mathbf{w}))^2 = \mathbb{E}_{\mathcal{D}} \left[(v^\pi - \hat{v}(s; \mathbf{w}))^2 \right]$$

Experience Replay

SGD with Experience Replay

Given value function approximation $\hat{v}(s; \mathbf{w}) \approx v_\pi(s)$ and experience $\mathcal{D}$ consisting of $\langle state, value \rangle$ pairs

$$\mathcal{D} = \{ \langle S_1, v_1^\pi \rangle, \langle S_2, v_2^\pi \rangle, \dots, \langle S_T, v_T^\pi \rangle \}$$

Repeat

1. Sample state, value from experience

$$\langle s, v^\pi \rangle \sim \mathcal{D}$$

2. Apply stochastic gradient descent update

$$\Delta \mathbf{w} = \alpha (v^\pi - \hat{v}(s; \mathbf{w})) \nabla_{\mathbf{w}} \hat{v}(s; \mathbf{w})$$

Converges to least squares solution

$$\mathbf{w} = \arg \min_{\mathbf{w}} LS(\mathbf{w})$$

Deep Q-Networks (DQN)

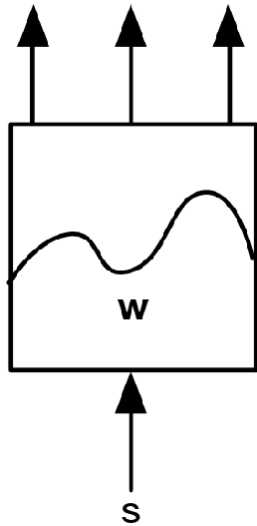
DQN uses **experience replay** and **fixed Q-targets**

- Take action a_t according to ϵ -greedy policy
- Store transition $(s_t, a_t, r_{t+1}, s_{t+1})$ in replay memory $\mathcal{D}$
- Sample random **mini-batch** of transitions (s, a, r, s') from $\mathcal{D}$
- Compute Q-learning targets with respect to old fixed parameters w^-
- Optimise MSE between **Q-network** and **Q-learning targets**

$$\mathcal{L}_i(w_i) = \mathbb{E}_{s,a,r,s' \sim \mathcal{D}_i} \left[\left(r + \gamma \max_{a'} Q(s', a'; w_i^-) - Q(s, a; w_i) \right)^2 \right]$$

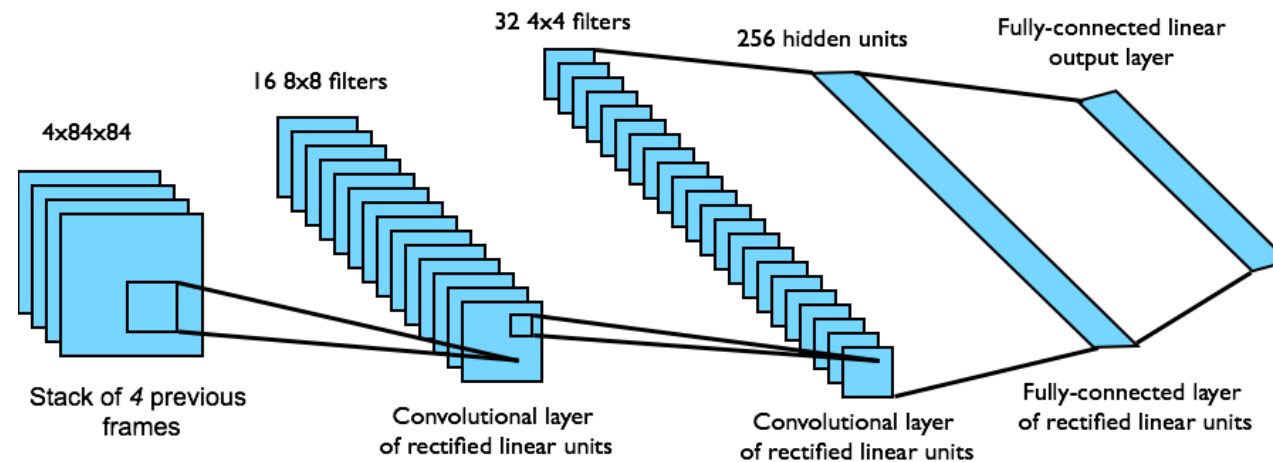
- Using variant of stochastic gradient descent

Atari-DQN

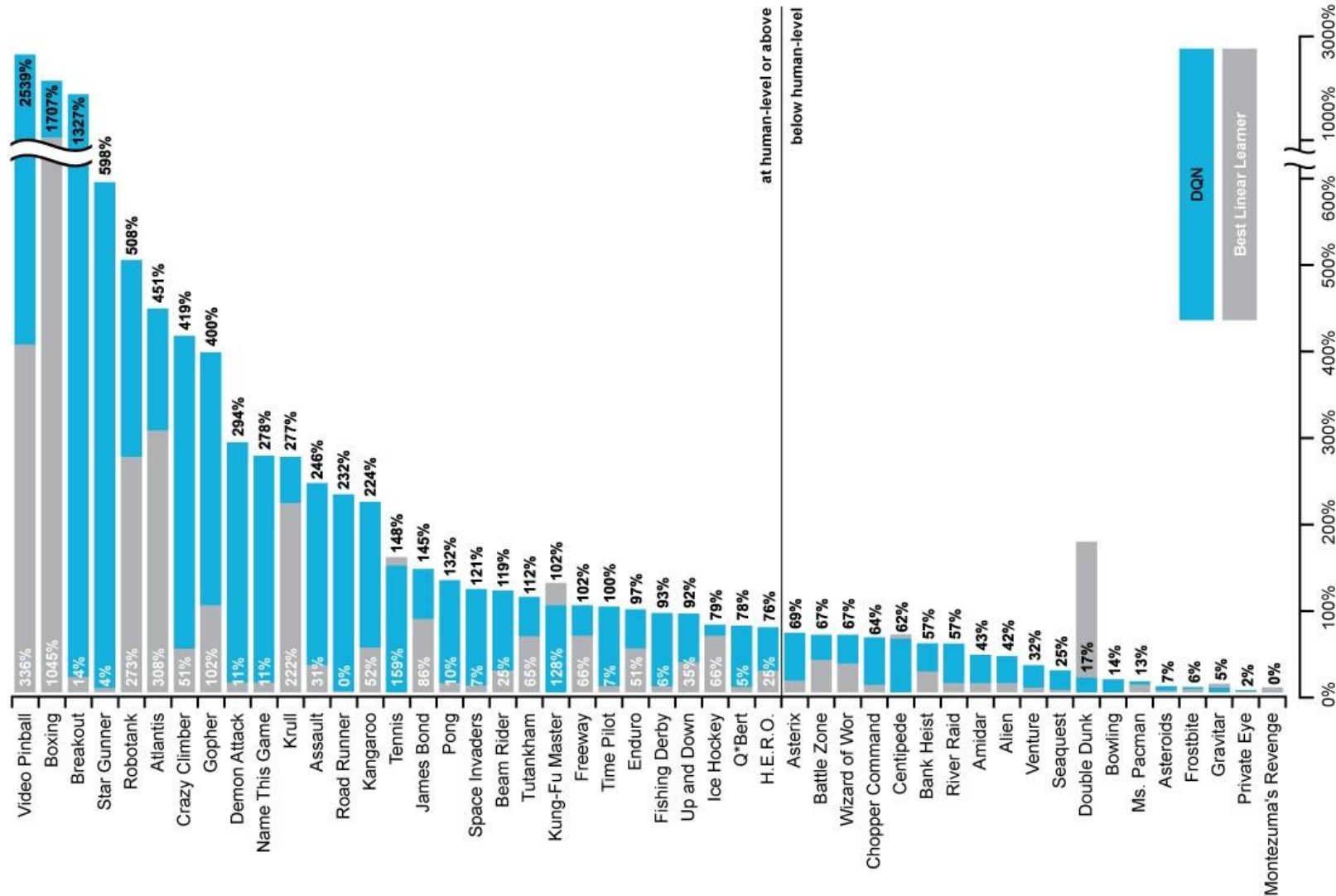


- End-to-end learning of values $Q(s, a)$ from pixels s
- Input state s is stack of raw pixels from last 4 frames
- Output is $Q(s, a)$ for 18 joystick/button positions
- Reward is change in score for that step

Network architecture and hyperparameters fixed across all games



Atari- DQN Results



Atari-DQN Ablation Study

	Replay Fixed-Q	Replay Q-learning	No replay Fixed-Q	No replay Q-learning
Breakout	316.81	240.73	10.16	3.17
Enduro	1006.3	831.25	141.89	29.1
River Raid	7446.62	4102.81	2867.66	1453.02
Seaquest	2894.4	822.55	1003	275.81
Space Invaders	1088.94	826.33	373.22	301.99

Improvements on Original DQN

Double DQN - Remove upward bias caused by $\max_a Q(s, a; \mathbf{w})$ -- i.e. solves the problem that we were picking the best action from an old copy of Q

- Current Q-network $\mathbf{w}$ is used to **select** actions
- Older Q-network $\mathbf{w}^-$ is used to **evaluate** actions

$$\mathcal{L}_i = \left(r + \gamma Q \left(s', \underset{a'}{\operatorname{arg\,max}} Q(s', a'; \mathbf{w}) ; \mathbf{w}^- \right) - Q(s, a; \mathbf{w}) \right)^2$$

Pick reward from old copy and pick action from new copy

Prioritised replay - Weight experience according to surprise (**prioritize samples with larger error**)

- Store experience in priority queue according to **DQN error**

Duelling network - Split Q-network into two channels

- Action-independent **value function** $V(s)$
- Action-dependent **advantage function** $A(s, a; \mathbf{w})$

$$Q(s, a) = V(s) + A(s, a; \mathbf{w})$$

In some states, whatever action is not relevant. Split V and A , and A can be computed only when matters

Least Squares Prediction and Control

Linear Least Squares Prediction

- Experience replay finds least squares solution, but it may take **many iterations**
- Using **linear value function** approximation $\hat{v}(s; \mathbf{w}) = \mathbf{x}(s)^T \mathbf{w}$
- We can solve the least squares solution directly

Linear Least Squares Prediction - Batch

- At minimum of $LS(\mathbf{w})$, the expected update must be zero

$$\mathbb{E}_{\mathcal{D}}[\Delta \mathbf{w}] = 0$$

$$\alpha \sum_{t=1}^T \mathbf{x}(S_t) (\mathbf{v}_t^{\pi} - \mathbf{x}(S_t)^T \mathbf{w}) = 0$$

$$\mathbf{w} = \left(\sum_{t=1}^T \mathbf{x}(S_t) \mathbf{x}(S_t)^T \right)^{-1} \sum_{t=1}^T \mathbf{x}(S_t) \mathbf{v}_t^{\pi}$$

- Usual direct (N^3) and incremental (N^2) costs

Linear Least Squares Prediction - Algorithms

- Again: we do not know the true v_t^π
- Training data use noisy or biased samples of v_t^π
 - Least Squares Monte-Carlo (**LSMC**) uses return $v_t^\pi \approx G_t$
 - Least Squares TD (**LSTD**) uses TD target $v_t^\pi \approx R_{t+1} + \gamma \hat{v}(S_{t+1}; \mathbf{w})$
 - Least Squares TD(λ) (**LSTD(λ)**) uses λ -return $v_t^\pi = G_t^\lambda$
- In each case solve directly for fixed point of MC / TD / TD(λ)

LS Updates

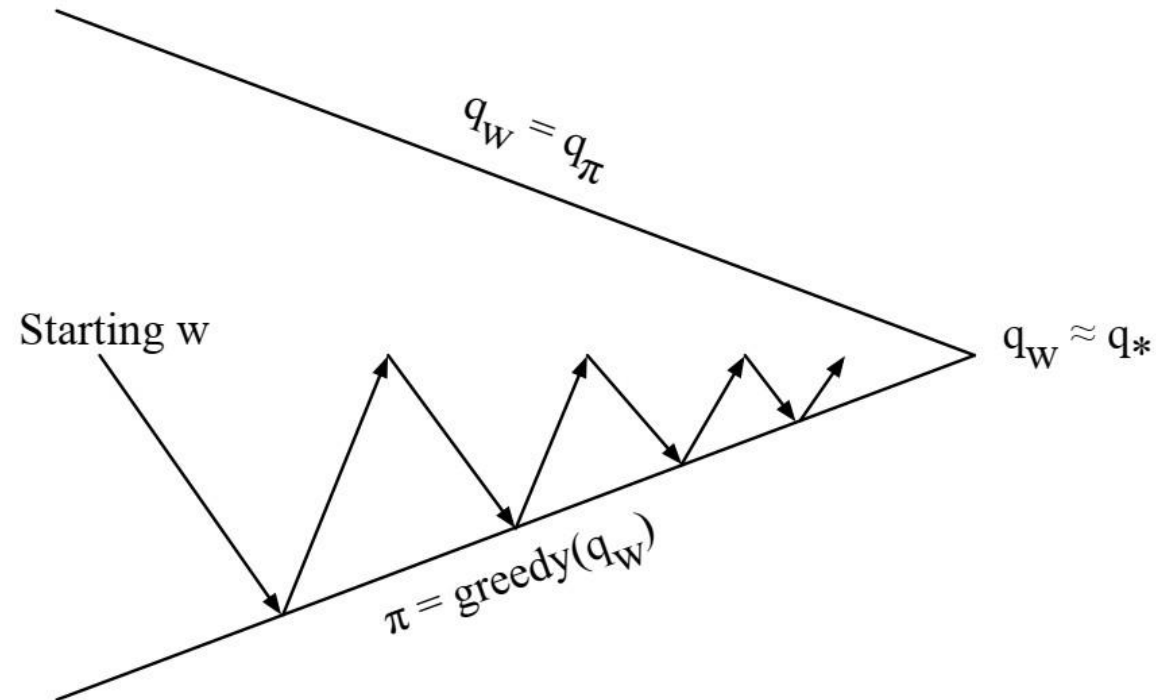
LSMC	$\mathbf{w} = \left(\sum_{t=1}^T \mathbf{x}(S_t) \mathbf{x}(S_t)^T \right)^{-1} \sum_{t=1}^T \mathbf{x}(S_t) G_t$
LSTD	$\mathbf{w} = \left(\sum_{t=1}^T \mathbf{x}(S_t) (\mathbf{x}(S_t) - \gamma \mathbf{x}(S_{t+1}))^T \right)^{-1} \sum_{t=1}^T \mathbf{x}(S_t) R_{t+1}$
LSTD(λ)	$\mathbf{w} = \left(\sum_{t=1}^T E_t (\mathbf{x}(S_t) - \gamma \mathbf{x}(S_{t+1}))^T \right)^{-1} \sum_{t=1}^T \mathbf{x}(S_t) R_{t+1}$

Solves one shot the problem of Least Square in linear case

Convergence of Linear Least Squares Prediction Algorithms

On/Off-Policy	Algorithm	Table Lookup	Linear	Non-Linear
On-Policy	MC	✓	✓	✓
	LSMC	✓	✓	-
	TD	✓	✓	X
	LSTD	✓	✓	-
Off-Policy	MC	✓	✓	✓
	LSMC	✓	✓	-
	TD	✓	X	X
	LSTD	✓	✓	-

Least Squares Control



- Policy evaluation - Policy evaluation by least squares Q-learning
- Policy improvement - Greedy policy improvement

Least Squares Action-Value Function Approximation

- Approximate $q_\pi(s, a)$ using linear combination of features $\mathbf{x}(s, a)$

$$\hat{q}(s, a; \mathbf{w}) = \mathbf{x}(s, a)^T \mathbf{w} \approx q_\pi(s, a)$$

- Minimise least squares error between $\hat{q}(s, a; \mathbf{w})$ and $q_\pi(s, a)$

- From **experience generated using policy π**

- Consisting of $\langle (state, action), value \rangle$ pairs

$$\mathcal{D} = \{ \langle (s_1, a_1), v_1^\pi \rangle, \dots, \langle (s_T, a_T), v_T^\pi \rangle \}$$

Least Squares Control

- For policy evaluation, we want to efficiently use all experience
- For control, we also want to improve the policy
- This experience is generated from many policies
- So to evaluate $q_\pi(S, A)$ we must learn off-policy
- We use the same idea as Q-learning:
 - Use experience generated by old policy $S_t, A_t, R_{t+1}, S_{t+1} \sim \pi_{old}$
 - Consider alternative successor action $A' \sim \pi_{new}(S_{t+1})$
 - Update $\hat{q}(S_t, A_t; \mathbf{w})$ towards value of alternative action $R_{t+1} + \gamma \hat{q}(S_{t+1}, A'; \mathbf{w})$

Least Squares Q-Learning

- Consider the following linear Q-learning update

$$\delta_t = R_{t+1} + \gamma \hat{q}(S_{t+1}, \pi(S_{t+1}); \mathbf{w}) - \hat{q}(S_t, A_t; \mathbf{w})$$

$$\Delta \mathbf{w}_t = \alpha \delta_t \mathbf{x}(S_t, A_t)$$

This is the target policy (greedy)

- LSTDQ algorithm: solve for total update equal to zero

$$\Delta \mathbf{w}_t = 0$$

$$\mathbf{w} = \left(\sum_{t=1}^T \mathbf{x}(S_t, A_t) [\mathbf{x}(S_t, A_t) - \gamma \mathbf{x}(S_{t+1}, \pi(S_{t+1}))]^T \right)^{-1} \sum_{t=1}^T \mathbf{x}(S_t, A_t) R_{t+1}$$

This comes from the behavior policy (e-greedy)

This is the target policy (greedy)

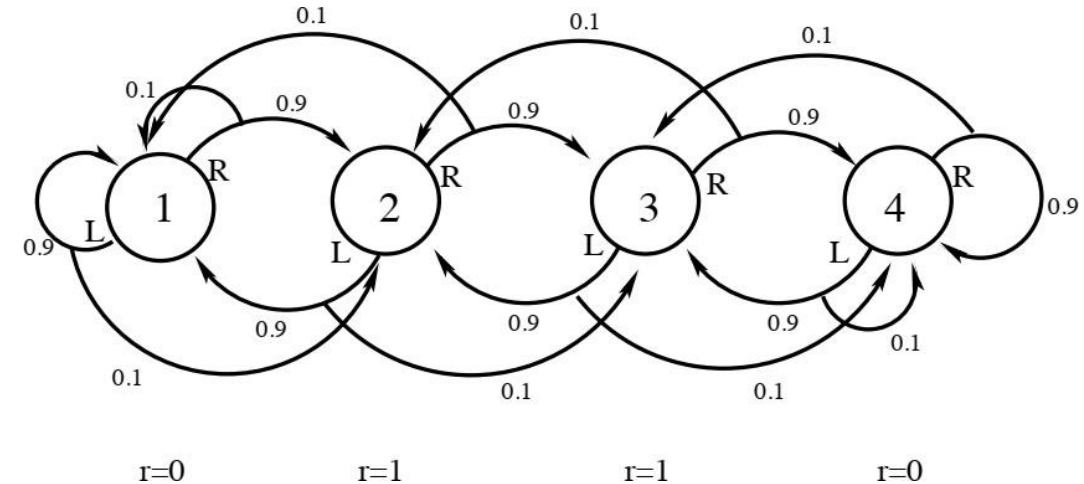
Convergence of Control Algorithms

Algorithm	Table Lookup	Linear	Non-Linear
Monte-Carlo Control	✓	(✓)	✗
Sarsa	✓	(✓)	✗
Q-learning	✓	✗	✗
LSPI	✓	(✓)	-

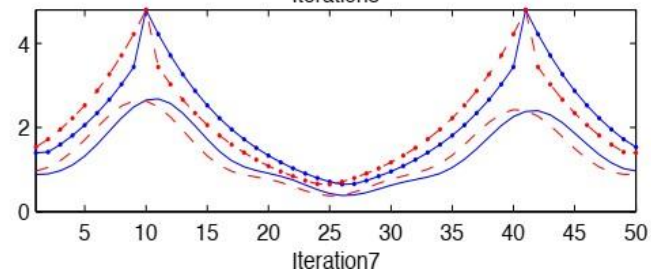
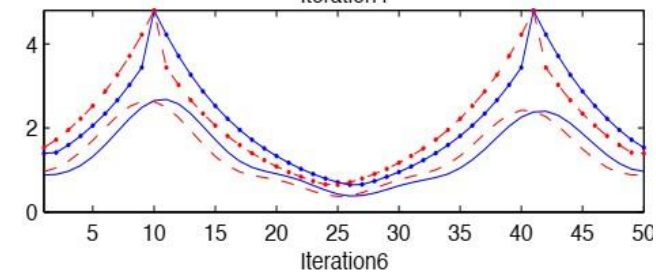
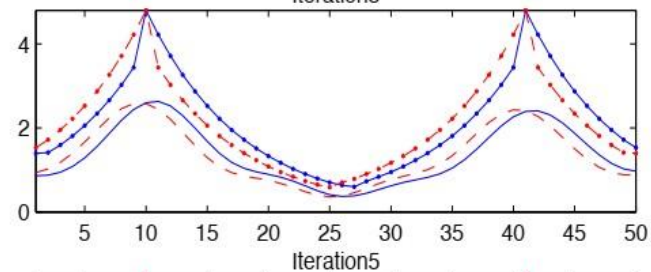
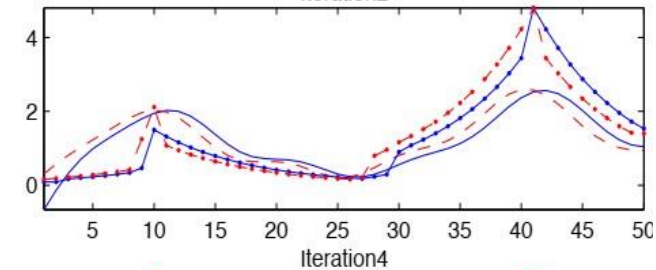
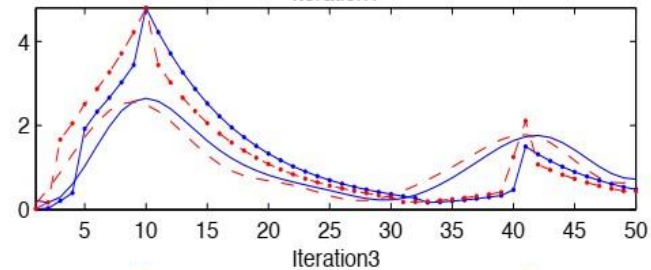
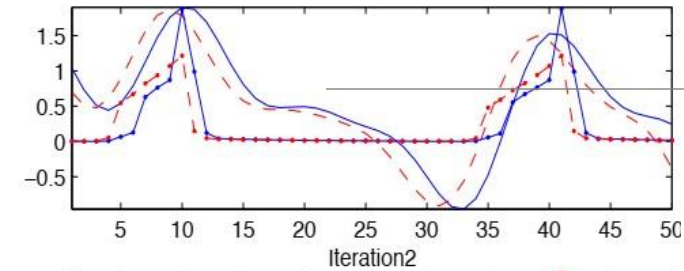
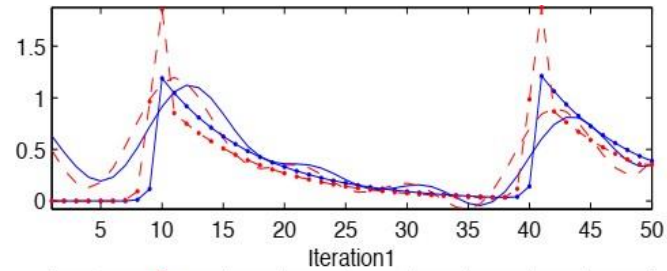
(✓) = chatters around near-optimal value function

Chain Walk Example

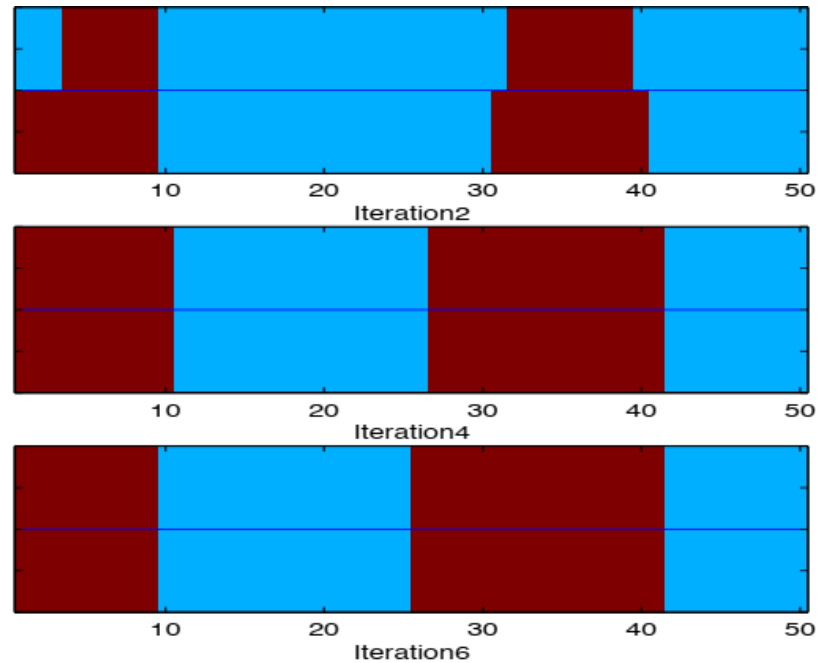
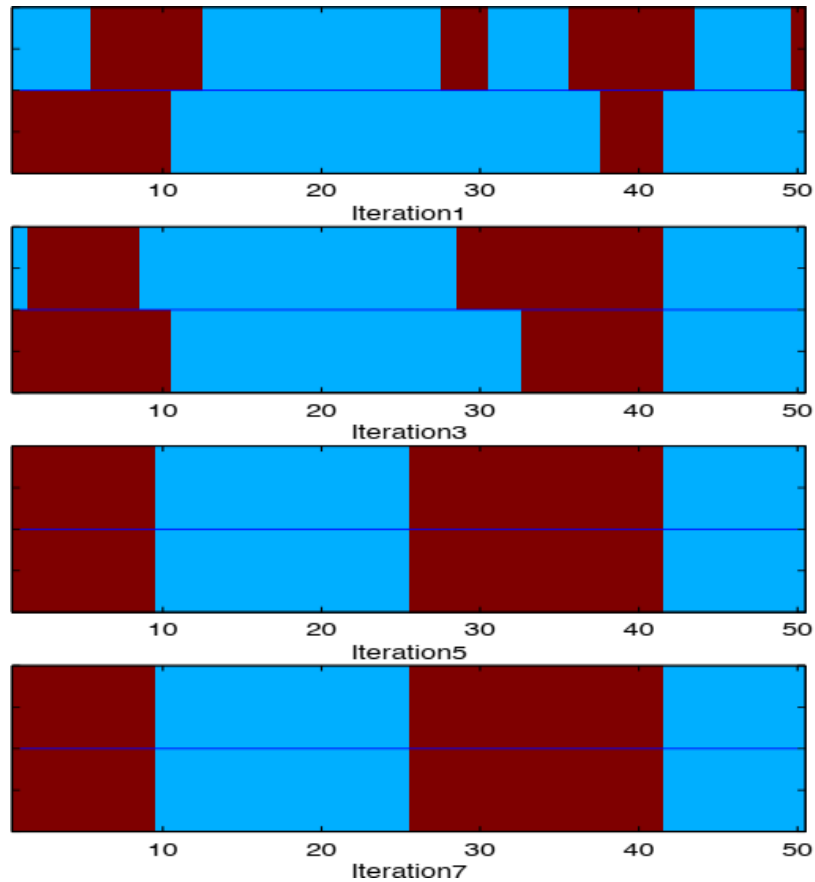
- ✓ Consider the 50 states version of this problem
- ✓ **Reward**: +1 in states 10 and 41, 0 elsewhere
- ✓ **Optimal policy**: R (1-9), L (10-25), R (26-41), L (42, 50)
- 41), L (42, 50)
- ✓ **Features**: 10 evenly spaced Gaussians ($\sigma = 4$) for each action
- ✓ **Experience**: 10,000 steps from random walk policy



LSPI in Chain Walk - Action-Value Function



LSPI in Chain Walk - Policy



Wrap-up

Take home messages

- Value function approximation: scaling up RL to real-world sized problems
- Use MC, TD and $TD(\lambda)$ to generate sample experiences for training the (action) value approximators
 - Stochastic Gradient Descent (incremental)
 - Least Squares (batch)
- Experience replay store data and updates with targets from old parameters (DQN)
- Linear least squares provides a closed form solution
- Convergence of learning might be tricky

Next Week

- Policy Gradients (I&II)
- Parameterise the policy NOT the value function
- REINFORCE (Monte-Carlo Policy Gradient)
- Actor-Critic approaches
- Natural gradient
- Deep policy networks
- Learning with continuous actions

